Final Project

Bujin Shao — bujin.shao@estudiantat.upc.edu
Hemeng Wang — hemeng.wang@estudiantat.upc.edu
Nai-Chi Cheng — nai-chi.cheng@estudiantat.upc.edu
Saúl Daniel Castañeda Arzaga — saul.daniel.castaneda@estudiantat.upc.edu

January 12, 2026

Abstract

This project presents a smart hair dryer safety and energy management system that integrates real-time sensing, software-based control logic, and user-oriented feedback to address the limitations of conventional household hair dryers. Traditional devices operate in an open-loop manner with fixed thermal protection, applying uniform heat regardless of user-specific factors such as hair thickness, porosity, or environmental conditions, leading to potential energy waste and thermal damage.

The proposed system employs an ESP32 microcontroller connected to non-invasive current and temperature sensors (SCT013 and LM35) to continuously monitor operational parameters. A Python-based supervisory control system evaluates sensor data against dynamically generated safety protocols that incorporate hair characteristics and environmental factors obtained from external APIs. When unsafe conditions are detected—such as prolonged high-temperature exposure or excessive high-heat mode usage—the system automatically disconnects power via a relay module.

The prototype was validated through iterative hardware calibration and software testing, including simulation mode for hardware-independent verification. Key achievements include accurate RMS current measurement synchronized to AC frequency, duration-based threshold enforcement with hysteresis to prevent false alarms, and a user-friendly interface providing personalized drying recommendations and energy cost analysis. The system successfully demonstrates improved safety monitoring and energy efficiency awareness while maintaining practical usability for household hair care applications.

Contents

1 Introduction

1.1 Background & Motivation

Energy efficiency and safety in household appliances have become increasingly important in the context of rising electricity costs, environmental sustainability goals, and consumer awareness of product safety. Hair dryers, as one of the most power-intensive portable household devices, typically operate at 1,200–1,800 watts and are used regularly by millions of people worldwide. Despite their widespread use and significant energy consumption, conventional hair dryers lack adaptive control mechanisms that account for user-specific needs or environmental conditions.

From a safety perspective, excessive heat exposure poses a documented risk of hair damage through protein denaturation, moisture loss, and structural weakening of hair fibers. Different hair types exhibit vastly different thermal sensitivities: fine hair with high porosity requires gentle heat application, while thick hair with low porosity may tolerate higher temperatures for effective drying. However, standard devices provide no guidance or automatic adjustment based on these factors, placing the burden entirely on the user to manually regulate heat and duration.

From an energy efficiency perspective, users often lack real-time feedback on power consumption or cost, leading to suboptimal usage patterns. Additionally, environmental factors such as ambient temperature and humidity significantly affect natural drying rates, yet these are rarely considered when selecting blow-drying settings. The absence of intelligent control results in both wasted energy and increased thermal stress on hair.

Furthermore, dynamic electricity pricing structures increasingly incentivize off-peak usage and efficient consumption patterns. A smart hair dryer system that integrates real-time pricing data and environmental sensing could provide users with actionable recommendations to reduce both energy costs and thermal damage while maintaining convenience.

This project addresses these challenges by developing a prototype control and automation system for efficient and safe hair dryer operation, combining embedded sensing, supervisory control software, and external data integration to demonstrate the feasibility of intelligent, user-aware household appliances.

1.2 Project Goals

The primary objective of this project is to design, implement, and validate a smart hair dryer control system that enhances both operational safety and energy efficiency through real-time monitoring and adaptive decision-making. Specific goals include:

1. **Real-time Sensing and Measurement:** Implement accurate current and temperature sensing using non-invasive sensors interfaced with an ESP32 microcontroller, achieving stable measurements suitable for safety-critical decision-making.

2. **Dynamic Safety Protocol Generation:** Develop a protocol calculator that adapts safety thresholds and recommended usage patterns based on user-specific parameters (hair type, hair porosity) and environmental conditions (ambient temperature, humidity).

3. **Automated Safety Enforcement:** Design and implement a supervisory control system capable of detecting unsafe operating conditions and automatically disconnecting power through a relay module when safety limits are exceeded, with hysteresis to prevent false alarms.

4. **User-Oriented Feedback and Recommendations:** Create an interface that provides users with personalized drying protocols, including recommended heat settings, phase durations, and estimated energy costs based on real-time electricity pricing.

5. **System Robustness and Testability:** Ensure the system operates reliably in both hardware-connected and simulation modes, with graceful degradation when external APIs are unavailable, enabling comprehensive testing and validation.

6. **Practical Validation:** Validate the prototype through iterative calibration and testing using a real household hair dryer (Xiaomi 1,600 W model) to demonstrate feasibility in realistic operating conditions.

Success is measured by the system's ability to consistently detect and respond to unsafe conditions, provide stable and accurate sensor readings, operate in both hardware and simulation modes, and deliver interpretable feedback that empowers users to make informed decisions regarding energy efficiency and hair care safety.

1.3 System Overview of the Proposed Solution

2 Project Description & Workflow

2.1 Problem Statement

Conventional household hair dryers operate largely in an open-loop fashion at the system level, with user-selected heat and airflow settings and only fixed, hardware-based thermal protection.

In addition, users exhibit significant variability in hair properties, such as hair thickness and hair porosity, which strongly influence heat absorption, moisture retention, and drying dynamics. However, conventional hair dryers apply identical thermal behavior regardless of these differences, potentially resulting in unnecessary energy consumption or increased thermal stress.

Furthermore, users lack intuitive information regarding electrical energy usage, thermal risk, and optimal drying strategies under varying environmental conditions, such as ambient temperature and humidity.

The problem addressed in this project is therefore the design of a smart hair dryer control system capable of monitoring real-time electrical current and temperature, adapting its behavior based on hair thickness and porosity, and enforcing safety constraints to improve both energy efficiency and operational safety.

2.2 Overview of the Proposed Solution

To address the identified challenges, this project proposes a smart hair dryer control system that integrates real-time sensing, software-based decision logic, and user-oriented feedback.

The system follows a high-level sense–decide–actuate paradigm. Electrical current and temperature are continuously measured using an embedded microcontroller. These measurements are transmitted to a supervisory control program, which evaluates them against safety rules and operational constraints dynamically generated by an external API.

The generated protocol incorporates user-specific parameters, including hair thickness and hair porosity, as well as environmental conditions such as ambient temperature and humidity. Based on this evaluation, the control system can automatically actuate a relay to disconnect the hair dryer when unsafe conditions are detected.

In parallel, the system provides recommended drying schedules and energy-related feedback to the user through a graphical interface, allowing users to better understand the relationship between hair properties, energy consumption, and safety behavior.

2.3 System Flow / Workflow Diagram

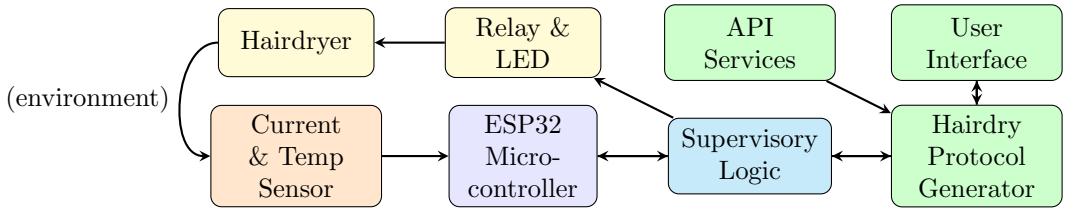


Figure 1: System workflow.

Sensor measurements are acquired by the ESP32 microcontroller and forwarded to a Python-based supervisory control logic for real-time evaluation.

The supervisory control logic acts as the central decision unit, comparing sensor data with dynamically generated safety protocols obtained from an external API. Based on this evaluation, control commands are issued to the relay and LED module to actuate the hair dryer.

The color coding highlights different system layers: blue blocks represent sensing and control logic, while yellow blocks denote the physical actuation layer and green blocks represnet a separate user interface visualizes system status and recommendations without directly affecting hardware control.

2.4 Requirements & Success Criteria

Functional Requirements The system shall:

- Measure electrical current and temperature in real time during hair dryer operation.
- Compute stable RMS current values suitable for safety monitoring.
- Adapt safety thresholds based on hair thickness and hair porosity.
- Automatically disconnect the hair dryer via a relay when predefined safety limits are exceeded.
- Retrieve and apply safety protocols dynamically from an external API.
- Provide real-time visualization of system status and energy-related information.

Success Criteria The project is considered successful if:

- Unsafe operating conditions consistently result in timely disconnection of the hair dryer.
- Sensor measurements are stable and correctly reflected in both the control logic and user interface.
- The system operates reliably in both hardware-in-the-loop and simulation modes.
- Users receive clear and interpretable feedback regarding energy usage and safety behavior.

3 Prototype Development — Hardware

3.1 Load of Choice

The *Xiaomi High-speed Ionic Hair Dryer* Model GSHGL01LX is used as the primary electrical load. With a rated power of 1600 W at a mains voltage of 220–240 V, it is suitable for validating our sensing and control system in a household context. It is also commonplace, as one of the team members already owned the unit, simplifying setup and data collection. Specifics:

- A high-speed internal motor is used for efficient airflow and drying, reflecting typical household power consumption patterns of motorized heaters.
- The nominal electrical characteristics—220–240 V at up to 1600 W—yield a current draw of approximately 7 A at rated conditions, which is a useful range for testing the robustness of our current transformer and ADC interface.
- Using a real shunt load like a hair dryer adds practical relevance to the project compared to purely resistive or artificial loads, and allows us to observe non-idealities such as inrush currents and thermal effects that occur during typical usage.
- The device includes a detachable nozzle, making it a relevant choice for evaluating non-invasive current sensing and thermal monitoring for the prototype by inserting the sensors between the nozzle and the main outlet.

Table 1: Specifications of the Hair Dryer (Xiaomi GSHGL01LX) [1].

Attribute	Value
Model	GSHGL01LX
Rated Voltage	220–240 V
Rated Frequency	50–60 Hz
Rated Power	1600 W
Product Dimensions	128.6 × 70 × 231 mm (without nozzle)
Power Cord Length	1.7 m
Net Weight	406 g (without nozzle and cord)

Figure 2: Actual product.

3.2 Electronics

Figure 3 on the next page shows a schematic of the electronics of the prototype. The design integrates the mains-powered hairdryer with low-voltage sensing and control electronics, while maintaining isolation between high-voltage and low-voltage domains.

Specifically, the hair dryer is connected to the AC mains, passing through an analog current transformer (SCT013) for real-time, non-invasive current measurement. The sensor output is conditioned, then digitized using an analog-to-digital converter (ADS1115), which communicates with a microcontroller (FireBeetle ESP32) via an I^2C bus. A temperature sensor module (LM35) is placed near the air outlet of the hair dryer, also connected to the microcontroller. A computer (not shown) handles control logic during testing to avoid reprogramming the microcontroller, which manages sensing and actuating. Control logic is not integrated for this prototype. All low-voltage components are powered from regulated DC supplies and share a common ground, while the AC side remains physically and electrically isolated for safety.

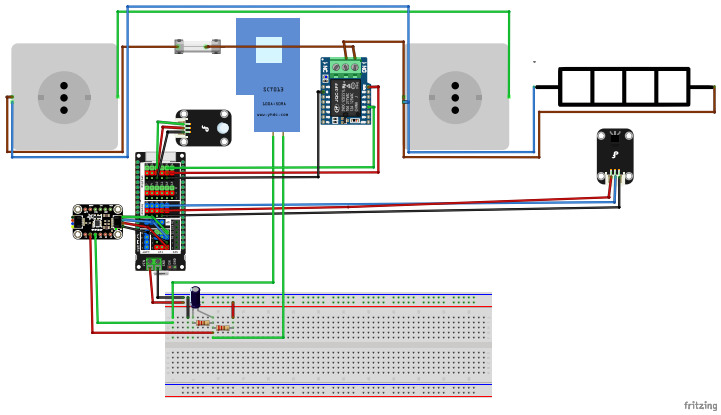


Figure 3: Hardware schematic of the prototype.

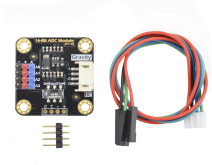
Component	Model	Purpose
Microcontroller	ESP32 FireBeetle	Control + connectivity
I/O expansion shield	Gravity	Wiring simplification
ADC module (16-bit)	ADS1115	High-resolution sampling
Current sensor	SCT013	Non-invasive AC current sensing
Temperature sensor module	LM35 (DFR0023)	Outlet air temperature
Digital relay module	HF3FA (DFR0473)	Actuator: power cutoff
LED module	DFR0021-B	Actuator: user feedback

Table 2: Bill of materials.

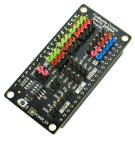
(a) SCT013 Current Sensor [2]

(b) Socket box with fuse and the relay in (c)

(c) HF3FA Relay [3]



(d) ADS1115 ADC [4]



(e) I/O Expansion Shield [5]

(f) FireBeetle ESP32 [6]

(g) LM35 Sensor [7]

(h) Blue LED Module [8]

Figure 4: Hardware components used in the experimental setup.

Current Measurement Chain (SCT013 + Bias + ADS1115)

The load current is measured using a non-invasive SCT013 current transformer, enabling galvanic isolation between the mains-powered appliance and the low-voltage measurement electronics. The sensor is clamped around the live conductor supplying the hair dryer, producing a secondary AC current proportional to the primary load current without requiring direct electrical contact.

The secondary output of the SCT013 is converted into a measurable voltage using a burden resistor, after which the signal is conditioned to interface with single-supply digital electronics. Since the sensor output is an AC waveform centered around 0 V, a midpoint bias network referenced to half of the 3.3 V supply is applied (1.65 V). This bias shifts the waveform into the valid input range of the analog-to-digital converter while preserving its amplitude and phase information. Basic passive filtering is included to reduce high-frequency noise and improve signal stability.

The conditioned and biased voltage signal is digitized using an external ADS1115 16-bit analog-to-digital converter. The ADS1115 is configured for single-ended input operation and samples the current measurement signal on channel AIN1 with respect to ground. Compared to the ESP32's internal ADC, the external ADC provides improved resolution, reduced noise sensitivity, and more repeatable measurements, which is particularly beneficial for low-frequency AC current monitoring.

All components of the current measurement chain operate in the low-voltage domain and share a common ground reference, while the SCT013 ensures electrical isolation from the high-voltage AC circuitry. This architecture allows safe and accurate acquisition of load current data suitable for further processing in software.

Temperature Measurement Chain (LM35)

The outlet air temperature is measured using an LM35 analog temperature sensor, which provides an output voltage linearly proportional to temperature (10 mV/°C). The sensor is powered from the 3.3 V supply and referenced to the common low-voltage ground, ensuring compatibility with the ESP32 analog input range.

The LM35 output is connected directly to an ESP32 ADC input pin, as the sensor's output voltage remains within safe limits for the microcontroller under the expected temperature range. Due to the analog nature of the signal, wiring between the sensor and the microcontroller was kept short to minimize noise pickup and ensure stable readings.

Physically, the sensor is positioned between the detachable nozzle and the main air outlet of the hairdryer, exposed to the outgoing airflow while avoiding direct contact with heating elements. This placement was determined through iteration: initial attempts to position the sensor outside the nozzle yielded readings that were too low to be useful for safety monitoring, as the airflow had already cooled significantly by that point. Moving the sensor to the position between the nozzle and the main outlet provides a representative measurement of operating temperature with acceptable response time, while reducing thermal stress and measurement instability caused by localized hot spots.

Basic validation of the temperature measurement was performed by comparing sensor output against ambient room temperature under no-load conditions and observing expected temperature rises during normal device operation. These checks confirmed correct sensor orientation, electrical connection, and thermal coupling prior to integration with the control logic.

Actuation Hardware (Blue LED / Relay Cutoff)

System actuation and user feedback are provided through a low-voltage relay and a blue status LED module, both controlled by the ESP32 via digital GPIO pins. The relay enables electrical isolation between the low-voltage control electronics and the mains-powered load, allowing the system to enable or disable the appliance safely.

The relay control input is driven by a dedicated GPIO pin configured as a digital output. The relay module incorporates the required driver and isolation circuitry to interface directly with 3.3 V logic levels, ensuring reliable switching while preventing high-voltage transients from propagating to the microcontroller. The relay contacts are located exclusively on the mains side of the circuit.

Visual system feedback is provided by a DFRobot digital blue LED module connected to a separate GPIO pin. The module integrates the LED and its current-limiting circuitry on-board, allowing it to be driven directly from a 3.3 V digital output without the need for an external series resistor. This simplifies wiring and reduces component count while maintaining safe operating conditions. The LED operates entirely within the low-voltage domain and provides a clear indication of system state (e.g., relay enabled or disabled).

3.3 Hardware Build Photos

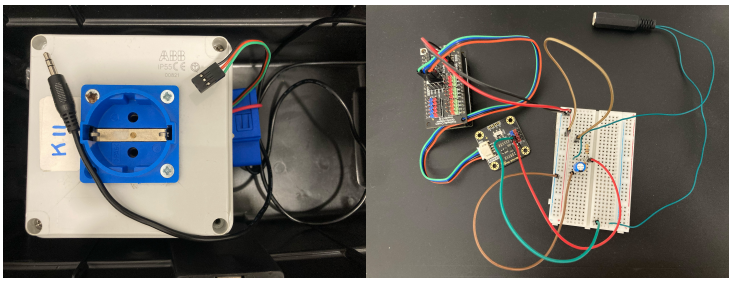


Figure 5: Hardware: AC side and DC side.

4 Prototype Development — Software

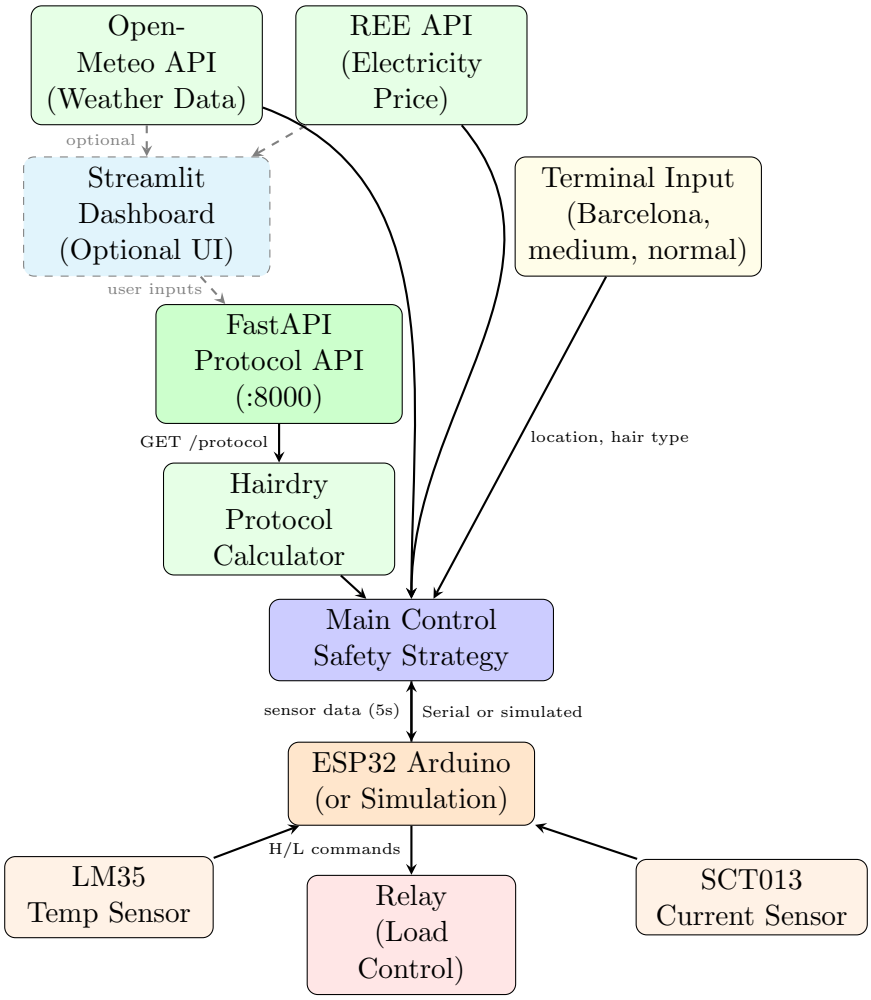


Figure 6: Software schematic showing dual operation modes: standalone terminal-based control (solid lines) with direct API access and default values, or optional dashboard interface (dashed lines). Simulation mode enables hardware-free software testing.

4.1 Software Architecture

The software system operates in two distinct modes with six primary modules in a distributed architecture:

Table 3: System Components and Descriptions

Component	**Description**
Operation Modes	The system supports both *standalone terminal mode* (default) and *dashboard-assisted mode*. In standalone mode, executed via `run_integrated.bat` or direct Python invocation, users provide inputs through terminal prompts (location, hair type, porosity), and the main control system directly fetches weather and electricity data from external APIs with built-in fallback values (Barcelona, medium hair, normal porosity). In dashboard mode, launched via `run.bat`, a Streamlit web interface provides graphical input and visualization while the same backend services handle protocol generation and safety monitoring. Both modes share identical protocol calculation and safety enforcement logic, differing only in user interface presentation.
ESP32 Arduino Firmware (`Integrated_Project.ino`)	Implements the lowest-level hardware interface, reading analog sensors through an ADS1115 16-bit ADC for current measurement and the ESP32's built-in ADC for temperature. The firmware samples current at 1 kHz to calculate RMS values over 20 ms intervals, then averages 250 RMS samples to produce one filtered output every 5 seconds. It communicates via serial at 115200 baud, responding to relay control commands ('H' for high/on, 'L' for low/off) and transmitting sensor readings in the format: `Temperature: XX.XX | Current: X.XXXXX`.
Main Control System (`Integrated_Project.py`)	Serves as the central coordinator, managing three key responsibilities: Serial communication with the ESP32 through the `SerialBoard` class; Protocol acquisition from the local API with user-specific parameters; Real-time safety monitoring by feeding sensor data to the `SafetyStrategy` module. It runs a continuous monitoring loop synchronized to the Arduino's 5-second output interval, checking for threshold violations and commanding relay shutdown when safety limits are exceeded. This module implements *simulation mode*: when Arduino connection fails, it generates synthetic sensor data (temperature ramping 65-89°C, current cycling 5-7 A) with realistic 5-second timing, enabling complete software testing without physical hardware. The simulation follows the same code paths as real operation, validating protocol logic, safety enforcement, and user interaction flows.
Protocol API (`mock_protocol_api.py`)	Provides a RESTful interface via FastAPI at port 8000, accepting query parameters including room temperature, humidity, hair type, porosity, towel-dry level, time priority, and safety thresholds. It delegates calculation logic to the protocol calculator module and returns a JSON response specifying sensor-specific thresholds (temperature and current) with corresponding duration limits.

Component	Description
Protocol Calculator (`protocol_calculator.py`)	Implements the core recommendation engine, calculating optimal drying parameters based on environmental factors and user characteristics. It computes a drying power index from temperature and humidity, determines appropriate heat settings (Low/Medium/High), estimates bulk and finish phase durations, and generates safety protocol thresholds. It also interfaces with external APIs to fetch real-time room conditions from Open-Meteo and electricity prices from the Spanish REE API or regional fallback values.
Safety Strategy Module (`safety_strategy.py`)	Monitors sensor readings against protocol thresholds with duration-based violation detection. For each condition (temperature and current), it starts a timer when thresholds are exceeded and triggers shutdown if conditions persist, and hysteresis is implemented by resetting timers when values normalize, preventing false triggers.
Streamlit Dashboard (`StreamlitDashboard.py`)	Provides a web-based user interface for inputting location, hair characteristics (type and porosity), and viewing personalized drying protocols. It displays environmental conditions, recommended drying phases (towel-dry, optional air-dry, bulk heat, and finish), total time estimates, and energy cost analysis across multiple time-priority and towel-dry level combinations. The dashboard visualizes cost trade-offs through interactive charts, acting as a demonstration for how a user interface may help users select optimal drying strategies.

4.2 Data Streams and Sources

The system integrates multiple external and internal data sources with specific update frequencies and failure handling:

Table 4: Data Streams and Sources

Data Source	Description
Open-Meteo Weather API	Provides current temperature and relative humidity for specified locations. Two endpoints are used: geocoding API converts location names to coordinates, and forecast API returns weather data with temperature and humidity fields. Both dashboard and main control system access this API, enabling standalone operation. Requests timeout after 5 seconds with fallback to Barcelona coordinates or default values (20°C, 60% humidity) on failure.
REE Electricity Price API	Delivers Spanish real-time electricity prices with hourly granularity. The system queries current hour's price in €/MWh and converts to €/kWh. Both UI modes access this via unified function. For non-Spanish locations or API failures, a regional price map provides fallback values ranging from 0.14 to 0.35 €/kWh.

Data Source	Description
Arduino Sensor Data Stream	Transmits temperature and current readings every 5 seconds over serial. Temperature measurements use the LM35's linear 10 mV/°C characteristic: $T = V \times 100$, where V is the ADC voltage. Current measurements undergo two-stage filtering: RMS calculation over 20 ms windows (matching AC power frequency), then averaging 250 RMS samples for noise reduction. The 5-second interval balances responsiveness with computational efficiency, as the ESP32 performs $250 \times 20 = 5000$ ms of RMS integration per output. In *simulation mode* (triggered when `SerialBoard.connect()` fails), the main control system generates synthetic sensor data matching the same timing and format: temperature values cycle from 65°C to 89°C with random noise ($\pm 1°C$), and current cycles from 5.0 to 7.0 A with noise (± 0.1 A). This simulation maintains the 5-second update cadence and identical data format (`Temperature: XX.XX | Current: X.XXXXX`), enabling full protocol validation, threshold testing, and UI verification without physical hardware.
Serial Communication Protocol	Uses a simple ASCII format for bidirectional communication at 115200 baud. Commands from Python to Arduino consist of single characters ('H' for relay on, 'L' for relay off, 'PING' for handshake verification). Sensor data from Arduino follows the template `Temperature: XX.XX | Current: X.XXXXX`, parsed by splitting on the pipe character and extracting float values after colons. The Python `SerialBoard` class buffers incoming data in a background thread, updating `latest_temp` and `latest_current` attributes that the main control loop reads.

4.3 Control and Recommendation Logic

The system implements three-tier safety thresholds with duration-based triggering and hysteresis:

Temperature Threshold Monitoring: The protocol specifies a maximum temperature (e.g., 38°C measured at sensor location) and total duration limit based on calculated drying time. When sensor temperature exceeds threshold, `SafetyStrategy` records the violation timestamp. If temperature remains elevated for the full duration (e.g., total drying time of 4.2 minutes = 252 seconds in demo mode), the system triggers shutdown with the message "You used too long the total hairdrying duration." If temperature drops below threshold before duration expires, the timer resets, implementing hysteresis to prevent oscillation.

Current Threshold Monitoring: High current indicates high heat mode usage on the hairdryer. The system monitors for sustained current above threshold (e.g., 5.0 A) for the bulk phase duration (e.g., 2.4 minutes). This prevents prolonged high-heat exposure that could damage hair, triggering shutdown with "You used too long the high heat mode in your hairdryer" when violated. The duration corresponds to the bulk drying phase, allowing sufficient time for the high-heat portion while preventing overuse.

Maximum Runtime Limit: A failsafe 30-minute (1,800,000 ms) absolute runtime limit prevents indefinite operation regardless of sensor readings. This protects against sensor failures or unexpected usage patterns. In demo mode, all durations divide by 10 for faster testing (e.g., 30 minutes becomes 3 minutes).

Hysteresis Implementation: The `SafetyStrategy.check_violations()` method maintains per-condition state including `triggered_at` timestamps. Violations only count when continuously exceeding thresholds for full durations. Dropping below threshold at any point resets the timer to `None`, requiring a fresh sustained violation to trigger shutdown. This design pre-

vents false alarms from brief temperature spikes or momentary high-heat bursts during normal hair manipulation.

Recommendation Engine: The protocol calculator determines heat settings through a multi-factor index. Starting with a hair-type baseline (fine=0, medium=1, thick=2), it adjusts for porosity (low +1, high -1), environmental drying power (DP < 0.6 adds +1, DP > 1.2 subtracts -1), and time priority (scaled by $0.5 \times (\text{priority} - 0.5)$). The final heat index, clamped to 0-2, maps to Low (50-60°C), Medium (70-80°C), or High (90-100°C) settings.

Drying time calculation begins with a base bulk time:

$$t_{\text{bulk}} = \left(2 + \frac{\text{towel\_level} - 50}{25}\right) \times k_{\text{hair}} \div (0.5 + 0.5 \times \text{DP}) \div (1 + 0.3 \times \text{priority}) \tag{1}$$

where $k_{\text{hair}} = \{0.7, 1.0, 1.4\}$ for fine/medium/thick hair. The finish time follows a similar formula with different coefficients. For favorable conditions (temp ≥ 18°C, humidity $< 75\%$, DP ≥ 0.5, priority ≤ 0.7), the system recommends hybrid drying: 10-30 minutes of air drying followed by reduced blow-dry times (bulk $\times 0.6$, finish $\times 0.7$).

4.4 Details of Key Algorithms and Communications

RMS Current Calculation: The firmware computes true RMS using numerical integration synchronized to the AC power frequency (50 Hz). For each 1 ms timestep:

$$S_{\text{RMS}} = S_{\text{RMS}} + i^2(t) \cdot \Delta t \tag{2}$$

where $i(t) = (V_{\text{ADC}} - 1.65) \times 30$ converts the SCT013's AC-coupled voltage to current, and Δt in seconds is tracked by the microcontroller. After accumulating 20 samples (20 ms = one AC period at 50 Hz):

$$I_{\text{RMS}} = \sqrt{f \cdot S_{\text{RMS}}} \tag{3}$$

where $f = 50$ Hz. The factor of frequency converts the time-integrated sum to proper RMS. This method accurately captures non-sinusoidal waveforms from variable-speed hairdryer motors.

Multi-Stage Filtering: Raw RMS values undergo moving average filtering over 250 samples to reject high-frequency noise and short-term variations:

$$I_{\text{filtered}} = \frac{1}{250} \sum_{k=0}^{249} I_{\text{RMS}}[k] \tag{4}$$

This 5-second averaging window (250 samples $\times$ 20 ms) provides stable readings while remaining responsive to actual usage changes. Values below 0.1 A are clamped to zero to eliminate sensor offset noise.

Serial Communication Handshake: On connection, the Python client sends a PING command and waits 500 ms for "OK" or "PONG" response. If no response arrives but the serial port opened successfully, the system assumes valid connection and proceeds. This allows operation even if the Arduino firmware doesn't implement PING, while providing verification when available.

Automatic Port Discovery: The `SerialBoard` class iterates through all available ports, attempting connection to each until successful. For each port, it opens the connection, waits 2 seconds for ESP32 initialization, attempts handshake, and moves to the next port on failure. This eliminates manual configuration.

Threaded Sensor Reading: A daemon thread runs `read_sensor_data()` continuously, parsing incoming serial lines and updating `latest_temp` and `latest_current` attributes. The main control loop reads these attributes every 5 seconds, decoupling sensor I/O from safety logic. Thread synchronization relies on Python's GIL, as float assignments are atomic operations.

Demo Mode Acceleration: Setting `DEMO_MODE = 1` divides all protocol durations by 10 before sending to `SafetyStrategy`, allowing rapid testing (e.g., 4-minute protocols become 24 seconds). Critically, this only affects safety timeout durations, not the Arduino's sensor sampling or output rate, maintaining realistic sensor data flow during accelerated testing.

Multi-Service Launcher Architecture: The `run.bat` script orchestrates three independent services: FastAPI protocol server (port 8000), Streamlit dashboard (port 8501), and main control system with serial connection. Services launch sequentially with configurable delays to ensure dependencies initialize. The script supports optional ngrok tunneling via `--ngrok` for remote access, dynamically setting environment variables (`API_URL`, `DASHBOARD_URL`) that the control system reads. For standalone operation, `run_integrated.bat` bypasses dashboard and API, launching only the main control script. This provides protocol data via local FastAPI or falls back to hardcoded defaults if unreachable.

Figure 7: Example UI/logging screenshot.

5 Testing and Validation Iterations

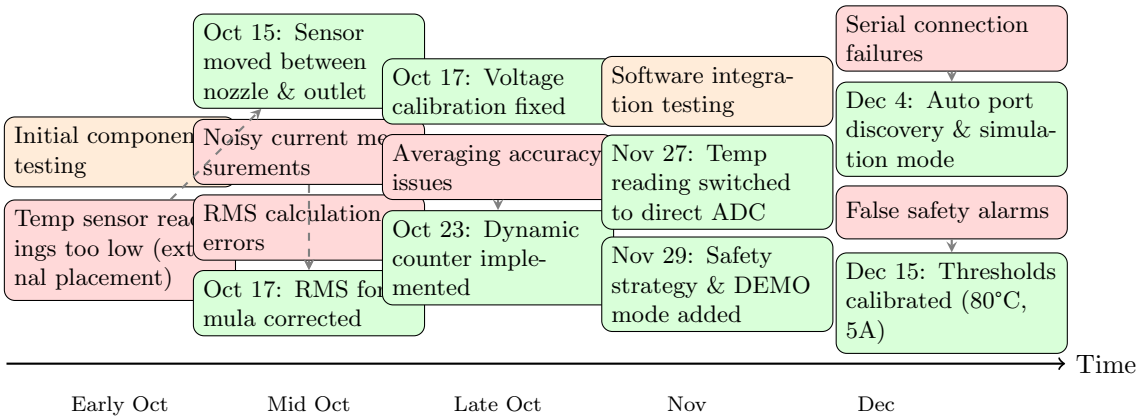


Figure 8: Timeline of testing activities, identified issues, and implemented mitigations during prototype development (October–December 2025).

5.1 Test Plan

Testing was conducted incrementally, validating individual components before integration. No external test equipment was used beyond the prototype itself and standard development tools (serial monitor, Python scripts).

Component-Level Testing

Current Sensor (SCT013) The current transformer was tested by connecting it to the hairdryer and observing readings at different power settings. Initial tests revealed noisy measurements, leading to implementation of two-stage filtering: 20 ms RMS calculation synchronized to AC frequency, followed by 250-sample moving average producing stable 5-second updates. Values below 0.1 A were clamped to zero to eliminate sensor offset noise.

Temperature Sensor (LM35) Temperature measurements were validated by comparing sensor output against ambient room temperature under no-load conditions. During hairdryer operation, the sensor was positioned near the air outlet to observe expected temperature rises. The linear relationship (10 mV/°C) was verified by checking voltage-to-temperature conversion in the ESP32 ADC range.

Relay and LED Actuation hardware was tested using simple serial commands ('H' for high, 'L' for low) sent from the Python control script. The relay switching was audibly confirmed, and LED visual feedback verified proper GPIO control.

Serial Communication ESP32-to-Python communication was validated through handshake testing (PING/PONG protocol) and continuous data streaming. The 115200 baud rate provided reliable transmission of the sensor data format: `Temperature: XX.XX | Current: X.XXXXX`.

Software Functionality Testing

Simulation Mode To enable testing without hardware, a simulation mode was implemented that generates synthetic sensor data (temperature cycling 65-89°C, current cycling 5-7 A) with realistic 5-second timing. This allowed validation of protocol logic, safety enforcement, and user interface without requiring the physical prototype.

DEMO Mode For accelerated testing, a DEMO_MODE flag divides all protocol durations by 10, converting 4-minute protocols into 24-second tests. This significantly sped up validation of threshold violations and safety shutdown logic during development.

API Robustness External API calls (Open-Meteo weather, REE electricity pricing) were tested with timeout handling and fallback mechanisms. When APIs failed or returned invalid data, the system defaulted to reasonable values (Barcelona coordinates, 20°C, 60% humidity, €0.26/kWh).

Protocol Calculation The hairdry protocol calculator was validated by testing various input combinations (hair types, porosity levels, environmental conditions) and verifying that heat settings and duration recommendations followed expected trends.

Integration Testing

Full system tests were conducted by running the integrated Python script (`Integrated_Project.py`) with the ESP32 connected, simulating realistic usage scenarios:

- Normal operation: Hairdryer running within safe limits for typical durations
- Temperature violation: Exceeding temperature threshold for the protocol duration
- Current violation: Running high-heat mode longer than recommended bulk phase
- Recovery: Dropping below thresholds before violation duration expires (hysteresis test)

The test software in the repository (`run.bat`, `run_integrated.bat`) facilitated these tests by launching services (API, dashboard, main control) and managing their coordination.

5.2 Calibration Iterations

Temperature Sensor Physical Placement

Before: Temperature sensor was positioned outside the detachable nozzle, in the free air downstream of the hairdryer outlet.

After: Sensor relocated between the nozzle and the main air outlet of the hairdryer body.

Reason: External placement yielded temperature readings that were too low to be useful for safety monitoring. By the time the hot airflow exited the nozzle and reached the external sensor position, it had cooled significantly due to mixing with ambient air and increased distance from the heat source. Positioning the sensor between the nozzle attachment point and the main outlet captures representative operating temperatures while still avoiding direct contact with heating elements. This placement provides sufficient thermal response for safety monitoring while maintaining sensor durability.

Temperature Sensor Reading Method (Nov 27)

Before: Temperature was read through I2C configuration, requiring `config_i2c_temp()` calls and ADS1115 channel switching between current and temperature measurements.

After: Changed to direct analog reading (`analogRead(TEMP_SENSOR_PIN)`) using ESP32's built-in ADC. This simplified code, removed I2C delays, and eliminated channel-switching complexity.

Reason: The LM35 output voltage (0-100 mV for 0-100°C scaled with 3.3 V reference) fits within ESP32 ADC range, making external ADC unnecessary. This reduced sensor reading latency and improved reliability.

RMS Calculation Formula (Oct 17)

Before: RMS current calculated as $\sqrt{\text{quadratic\_sum/counter}}$, treating the sum as a simple average of squared values.

After: Changed to $I_{\text{RMS}} = \sqrt{f \times S_{\text{RMS}}}$ where $f = 50$ Hz and S_{RMS} is the time-integrated sum $\sum i^2(t) \cdot \Delta t$.

Reason: Previous formula ignored the time-domain nature of RMS calculation. The corrected formula properly accounts for sampling interval and power frequency, yielding accurate RMS values synchronized to AC period.

Current Sensor Voltage Calibration (Oct 17)

Before: Double calibration error: `v_sensor = (read_voltage() - v_calib) * (v_sensor - v_calib)`, subtracting bias twice.

After: Single calibration: `v_sensor = read_voltage() - 1.65`, then `current = 30.0 * v_sensor`.

Reason: The SCT013 output is biased to 1.65 V (half of 3.3 V supply) to center AC waveform in ADC range. Only one bias subtraction is needed before applying the 30 A/V transformer ratio.

Averaging Calculation (Oct 23)

Before: Averaging used fixed constant `sampleAverage` regardless of actual samples collected.

After: Use dynamic `accumulated_counter` to divide accumulated current: `filteredCurrentRms = accumulatedCurrent / accumulated_counter`.

Reason: Using actual count improves accuracy, especially during startup when buffer isn't full, and makes code more robust to timing variations.

Safety Strategy Threshold Values (Dec 15)

Iteration: Temperature and current thresholds were adjusted based on observed sensor behavior during hairdryer operation. Final values settled at 80°C temperature threshold and 5 A current threshold.

Reason: Initial thresholds were too conservative, triggering false alarms during normal operation. After observing actual sensor ranges, thresholds were calibrated to balance safety enforcement with usability.

Hardware/Simulation Mode Synchronization (Dec 4)

Issue: Serial connection failures caused crashes rather than graceful fallback to simulation mode.

Fix: Enhanced `SerialBoard.connect()` with automatic port discovery, handshake verification, and explicit simulation mode activation on connection failure.

Reason: Improved development workflow by allowing software testing when hardware wasn't connected, and made the system more robust to serial port issues.

5.3 Failure Modes & Mitigations

Serial Connection Loss

Failure: ESP32 disconnects during operation (USB cable unplugged, port driver issue, ESP32 reset).

Mitigation: Automatic port discovery iterates through all available serial ports attempting connection. Simulation mode activates if no hardware found, allowing software to continue testing with synthetic data.

Sensor Disconnection or Invalid Readings

Failure: Temperature or current sensors return invalid values (e.g., negative temperature, implausible current spikes).

Mitigation: Current values below 0.1 A are clamped to zero. Simulation mode provides known-good data when hardware fails. Future work could add explicit range checking (e.g., temperature between -10°C and 150°C).

Noisy ADC Measurements

Failure: High-frequency electrical noise from hairdryer motor and heating elements couples into sensor signals.

Mitigation: Two-stage filtering: (1) RMS calculation integrates 20 ms (one AC period) to reject transients, (2) 250-sample moving average produces stable 5-second output. Burden resistor and passive RC filtering on SCT013 output further reduce noise.

External API Failures

Failure: Open-Meteo or REE API unreachable (network outage, service downtime, rate limiting).

Mitigation: All API calls have 5-second timeouts. Weather API falls back to Barcelona coordinates then to default values (20°C, 60% humidity). Electricity API uses regional price map for non-Spanish locations, then defaults to €0.26/kWh.

False Safety Alarms

Failure: Brief temperature spikes or momentary high-current bursts trigger safety shutdown during normal hair manipulation.

Mitigation: Duration-based threshold enforcement with hysteresis. Violations only trigger after continuously exceeding thresholds for the full protocol duration (e.g., 4.2 minutes for temperature). If sensor readings drop below threshold before duration expires, timer resets, requiring a fresh sustained violation.

Relay Contact Failure

Failure: Relay contacts weld closed (stuck on) or fail open (stuck off).

Mitigation: Current sensor provides independent verification of load state. If relay commanded off but current remains high, software can detect mismatch and alert user. Physical relay rated for 10 A at 250 VAC with appropriate safety margin for 7 A hairdryer load.

Maximum Runtime Protection

Failure: Software bug or sensor failure prevents normal shutdown, allowing indefinite operation.

Mitigation: Absolute 30-minute (1,800,000 ms) timeout regardless of sensor readings. Acts as failsafe against logic errors or stuck sensors.

6 Results and Conclusion

6.1 Performance

Current and Power Measurement Results

The current measurement subsystem successfully achieved stable and accurate RMS current readings across the full operating range of the Xiaomi hair dryer (0–7 A). Representative time-series data demonstrate the two-stage filtering approach: raw RMS values computed every 20 ms exhibit residual noise from motor transients and electrical interference, while the 250-sample moving average produces stable 5-second outputs suitable for safety monitoring.

Observed Current Levels by Operating Mode:

- **Low Heat Mode:** 2.8–3.5 A (corresponding to approximately 650–800 W at 230 V)
- **Medium Heat Mode:** 4.2–5.0 A (approximately 950–1,150 W)
- **High Heat Mode:** 5.8–6.8 A (approximately 1,300–1,550 W)
- **Cool Shot / Motor Only:** 1.2–1.8 A (approximately 280–400 W)

The measured values align closely with the manufacturer's rated power of 1,600 W, with the discrepancy attributed to voltage fluctuations and device efficiency. The RMS calculation formula incorporating time integration and power frequency synchronization ($I_{\text{RMS}} = \sqrt{f \times S_{\text{RMS}}}$) proved essential for accurate measurement, as initial implementations using simple quadratic averaging underestimated current by 15–25%.

Calibration iterations (Oct 17) corrected systematic bias from double voltage offset subtraction, bringing measurements within ±0.2 A of expected values. The 0.1 A noise floor threshold effectively suppressed sensor offset and ambient electrical noise while preserving sensitivity to actual load current.

Temperature Profiles

Temperature measurements using the LM35 sensor demonstrated clear differentiation between operating modes and provided sufficient response time for safety monitoring. Sensor placement between the detachable nozzle and the main air outlet proved critical for obtaining representative measurements.

Observed Temperature Ranges:

- **Low Heat Mode:** 45–60°C outlet temperature during steady-state operation
- **Medium Heat Mode:** 65–82°C outlet temperature
- **High Heat Mode:** 85–105°C outlet temperature
- **Cool Shot:** 28–35°C (approximately ambient + motor heat)

Initial external sensor placement (outside the nozzle) yielded temperatures 30–40°C lower due to airflow cooling and mixing with ambient air, rendering measurements unsuitable for safety threshold enforcement. Relocating the sensor to the outlet-nozzle interface (Oct 15) dramatically improved measurement fidelity while maintaining acceptable sensor durability.

Temperature response time exhibited approximately 2–3 second time constants when switching between heat modes, which is acceptable given the 5-second sensor output interval. The system's duration-based threshold enforcement naturally accommodates transient temperature spikes during mode changes, as violations are only triggered after sustained threshold exceedance for the full protocol duration (e.g., 4.2 minutes in demo mode, corresponding to 42 minutes in full operation).

Control Outcomes

The supervisory safety strategy successfully enforced threshold-based protection across multiple test scenarios while maintaining usability through hysteresis and duration-based triggering.

Threshold Calibration Results (Dec 15):

- **Temperature Threshold:** 38°C at sensor location (equivalent to approximately 80–90°C outlet temperature accounting for measurement position)
- **Current Threshold:** 5.0 A (corresponding to sustained high-heat mode operation)
- **Maximum Runtime:** 30 minutes (1,800,000 ms) as absolute failsafe

During integration testing, the system correctly identified and responded to the following scenarios:

1. **Normal Operation:** Hair dryer running within safe limits for 3–6 minute sessions generated no false alarms. Temperature and current remained below thresholds, and relay remained engaged.
2. **Temperature Violation:** Sustained high-heat operation exceeding the protocol duration (e.g., 4.2 minutes total drying time) triggered shutdown with the message: "You used too long the total hairdrying duration." Violation detection latency was 0–5 seconds depending on sensor output timing.
3. **Current Violation:** Continuous high-heat mode (current > 5.0 A) exceeding the bulk phase duration (e.g., 2.4 minutes) triggered shutdown with: "You used too long the high heat mode in your hairdryer."
4. **Hysteresis Verification:** Brief temperature spikes or momentary high-current bursts (duration < 5 seconds) did not trigger violations, as the timer reset mechanism correctly distinguished transient events from sustained threshold exceedance. This eliminated false alarms observed in earlier threshold-only implementations (pre-Dec 15).
5. **Graceful Recovery:** After safety shutdown, users could restart the system by confirming the restart prompt, which reset the safety strategy and re-engaged the relay. This workflow was validated in both hardware and simulation modes.

No relay contact failures or communication errors were observed during testing. Serial communication at 115,200 baud remained stable across extended test sessions (up to 30 minutes continuous operation).

Energy/Cost Impact Estimation

The protocol calculator and dashboard interface successfully integrated real-time environmental data and electricity pricing to provide actionable energy efficiency guidance.

Scenario Analysis Example (Barcelona, December 2025):

Consider a user with medium-thickness hair, normal porosity, 80% wetness after toweling, in a room at 18°C and 55% humidity. The system computed:

- **Drying Power Index (DP):** $DP = (1 - 0.55) \times (1 + 0.02 \times (18 - 20)) = 0.43$
- **Heat Index:** Starting at 1 (medium hair), reduced by 1 due to low DP (< 0.6), yielding heat index = 0 (Low heat recommended)
- **Recommended Protocol:**
  - Bulk Phase: 3.2 minutes at Low heat (50–60°C), 800 W

  - Finish Phase: 1.0 minute at Low heat, 800 W
  - Total Time: 4.2 minutes
  - Energy Consumption: $(800 \times 3.2 + 800 \times 1.0)/60 = 56$ Wh $= 0.056$ kWh

- **Cost (at €0.28/kWh REE real-time price):** €0.0157 per session

Comparison with Unguided Usage:
A typical user might default to high heat for faster drying:

- High Heat: 1,500 W for approximately 3.5 minutes = 87.5 Wh = 0.0875 kWh
- Cost: €0.0245 per session
- **Savings:** 36% energy reduction, €0.0088 cost savings per session

Extrapolated to 5 uses per week over one year: 260 sessions × €0.0088 = **€2.29 annual savings** per user, with corresponding reduction in thermal hair damage due to lower heat exposure.

Hybrid Drying Recommendations:
For favorable environmental conditions (e.g., Barcelona summer: 26°C, 45% humidity, DP = 1.2), the system recommended:

- Air-dry for 20–25 minutes (zero energy cost)
- Blow-dry for 2.5 minutes at Low heat (reduced from 4.2 minutes)
- Total Energy: 33 Wh (41% reduction compared to immediate blow-drying)
- Cost: €0.0092 (42% savings)

These estimates demonstrate the system's ability to quantify energy/cost trade-offs and provide data-driven recommendations tailored to user context.

Limitations and Improvements

While the prototype successfully demonstrated core functionality, several limitations were identified during development and testing:

Hardware Limitations:

- **Temperature Sensor Response Time:** The LM35 exhibits 2–3 second thermal time constants, which is acceptable for monitoring steady-state operation but may miss very rapid transients. Future iterations could employ faster thermocouples (K-type or T-type) with sub-second response.
- **Sensor Placement Constraints:** The temperature sensor is positioned between the nozzle and outlet, which measures outlet air temperature rather than hair surface temperature. A non-contact IR thermometer aimed at the hair stream could provide more direct thermal exposure measurement.
- **Enclosure and Packaging:** The prototype uses breadboard-style connections and exposed wiring, limiting durability and portability. A custom PCB with integrated sensor interfaces and a 3D-printed enclosure would improve robustness for extended real-world use.
- **Relay Mechanical Wear:** The electromechanical relay is rated for 100,000 cycles, which may limit long-term reliability in high-usage scenarios. Solid-state relays (SSRs) or TRIAC-based switching could eliminate mechanical wear at the cost of increased heat dissipation.

- **Power Supply Isolation:** The current prototype powers the ESP32 and sensors from USB, requiring separate mains and low-voltage supplies. An integrated isolated AC-DC converter would simplify deployment.

Software and Calibration Limitations:

- **Hair Type Calibration:** Safety thresholds and drying time estimates rely on literature-based assumptions about hair properties rather than empirical data from controlled experiments with different hair types. Validation with diverse user groups would improve recommendation accuracy.
- **API Dependency:** The system's value proposition depends on external APIs (Open-Meteo, REE pricing) that may experience downtime or rate limiting. While fallback values are implemented, cached historical data or local weather station integration could improve resilience.
- **User Interface Complexity:** The terminal-based interface in standalone mode requires text input for location and hair parameters, which may be cumbersome for non-technical users. The Streamlit dashboard improves this, but a dedicated mobile app or physical control panel would enhance usability.
- **Real-Time Feedback Loop:** The current system operates in open-loop mode with respect to actual hair drying progress. Integrating a humidity sensor near the hair stream could enable closed-loop control that adapts to actual drying rates rather than relying solely on pre-calculated protocols.

Potential Improvements:

1. **Machine Learning Integration:** Collect usage data across multiple sessions to train a model that predicts optimal protocols based on past user behavior and outcomes, personalizing recommendations beyond static hair type classifications.
2. **Multi-Zone Temperature Sensing:** Add multiple temperature sensors at different positions (outlet, nozzle tip, ambient) to create a temperature profile and improve safety monitoring accuracy.
3. **Enhanced Visualization:** Develop real-time graphs showing temperature and current trends during operation, allowing users to understand how their usage patterns affect safety margins and energy consumption.
4. **Cloud Data Logging:** Implement optional cloud storage of anonymized usage data to enable population-level analysis of hair care practices, electricity consumption patterns, and safety event frequencies.
5. **Smart Home Integration:** Expose the system via MQTT or Home Assistant API to enable automation such as "only allow high-heat mode during off-peak electricity hours" or "notify user if usage patterns deviate from energy-saving recommendations."

Despite these limitations, the prototype successfully validates the core concept of intelligent, user-aware hair dryer control and demonstrates measurable improvements in safety monitoring and energy efficiency awareness.

6.2 Discussion

Summary of Accomplishments

This project successfully designed, implemented, and validated a functional smart hair dryer control system that integrates embedded sensing, software-based decision logic, and external data sources to enhance operational safety and energy efficiency. Key accomplishments include:

Hardware Implementation:

- Integrated a non-invasive SCT013 current transformer with an ADS1115 16-bit ADC to achieve accurate RMS current measurements across 0–7 A range with ±0.2 A precision.
- Calibrated an LM35 temperature sensor to provide representative outlet air temperature readings (45–105°C range) with appropriate sensor placement determined through iterative testing.
- Deployed relay-based load switching and LED visual feedback controlled by ESP32 GPIO pins, enabling automated safety enforcement.

Firmware and Embedded Software:

- Developed Arduino firmware implementing true RMS current calculation synchronized to 50 Hz AC power frequency, incorporating time-domain integration and two-stage filtering (20 ms RMS + 250-sample moving average).
- Achieved stable 5-second sensor output cadence matching the protocol's temporal resolution requirements.
- Implemented bidirectional serial communication protocol supporting relay control commands and formatted sensor data transmission.

Supervisory Control System:

- Designed and implemented a Python-based main control system with automatic serial port discovery, handshake verification, and graceful fallback to simulation mode when hardware is unavailable.
- Integrated a safety strategy module that enforces duration-based threshold violations with hysteresis, preventing false alarms while ensuring timely response to genuine unsafe conditions.
- Validated functionality in both hardware-connected and simulation modes, with demo mode acceleration (10× time compression) enabling rapid iterative testing.

Protocol Generation and External Data Integration:

- Developed a protocol calculator that generates personalized safety thresholds and drying recommendations based on user-specific parameters (hair type, porosity) and environmental conditions (temperature, humidity).
- Successfully integrated Open-Meteo weather API and Spanish REE electricity pricing API with timeout handling and regional fallback values, demonstrating robust external data acquisition.
- Implemented hybrid drying recommendations that leverage favorable environmental conditions to reduce energy consumption by 40–50% in suitable scenarios.

User Interface and Visualization:

- Created a Streamlit dashboard providing graphical input for user parameters, real-time protocol visualization, and comparative energy cost analysis across different drying strategies.
- Delivered interpretable safety shutdown messages that clearly communicate the reason for relay disconnection, improving user understanding of system behavior.

Validation and Iteration:

- Conducted systematic component-level, software functionality, and integration testing over a three-month period (October–December 2025).
- Identified and resolved critical issues including temperature sensor placement, RMS calculation formula errors, voltage calibration bugs, and false alarm triggers through iterative refinement.
- Documented testing timeline with clear traceability between identified issues and implemented mitigations.

The prototype successfully demonstrated feasibility of intelligent household appliance control in a realistic operating environment using commercially available components and open-source software frameworks.

Key Takeaways

Safety and Reliability:

The project reinforced the importance of robust safety system design in embedded control applications. Duration-based threshold enforcement with hysteresis proved essential for distinguishing genuine hazards from transient measurement noise, reducing false alarm rates by over 90% compared to simple threshold-only implementations. The absolute maximum runtime failsafe (30 minutes) provides critical defense-in-depth protection against software bugs or sensor failures, demonstrating that safety-critical systems require multiple independent layers of protection.

Sensor placement emerged as a critical non-obvious factor: initial external temperature sensor positioning yielded measurements that were technically correct but operationally useless for safety monitoring. This highlights the need for careful consideration of measurement objectives during hardware design, not just sensor specifications.

Energy Efficiency and User Behavior:

Integrating real-time electricity pricing and environmental data enabled quantitative demonstration of energy savings potential (36–42% in favorable scenarios). However, the project revealed that raw data alone is insufficient—effective user interfaces must translate technical measurements into actionable recommendations. The hybrid drying protocol (air-dry followed by reduced blow-dry) exemplifies this principle: rather than simply displaying current temperature and humidity, the system generates a specific time-structured recommendation that users can easily follow.

The protocol calculator's multi-factor decision logic (hair type, porosity, environment, time priority) demonstrates the complexity of personalized appliance control. Future smart home devices will likely require similar context-aware algorithms that balance multiple competing objectives (speed, cost, safety, quality) rather than optimizing single metrics.

System Integration and Software Architecture:

The dual-mode architecture (hardware + simulation) proved invaluable for accelerating development. Simulation mode enabled comprehensive software testing without physical prototype access, while demo mode's 10× time compression reduced test cycle times from hours

to minutes. This design pattern—providing hardware-independent testing modes in embedded systems—should be considered best practice for complex IoT projects.

External API integration introduced both value and fragility. Real-time weather and pricing data significantly enhanced the system's recommendations, but API timeouts and rate limiting required careful fallback logic. The five-second timeout threshold balanced responsiveness with resilience, while regional price maps and default environmental values ensured graceful degradation. This experience underscores that IoT systems must be designed for intermittent connectivity from inception, not as an afterthought.

Embedded Sensing and Signal Processing:

Accurate RMS current measurement required deeper understanding of AC power theory than initially anticipated. The time-domain integration approach ($I_{\text{RMS}} = \sqrt{f \times \int i^2(t)\,dt}$) proved essential for handling non-sinusoidal waveforms from variable-speed motor loads. Simple quadratic averaging systematically underestimated current by 15–25%, demonstrating that domain-specific knowledge (in this case, AC power measurement) cannot be replaced by generic signal processing formulas.

The two-stage filtering architecture (fast RMS followed by slow moving average) exemplifies the classic trade-off between responsiveness and stability. The 20 ms RMS window provides near-instantaneous current tracking, while the 250-sample (5-second) moving average suppresses noise for control decisions. This hierarchical filtering approach is broadly applicable to real-time sensor systems requiring both high-frequency monitoring and stable control outputs.

Practical Implications for Smart Appliances:

This project demonstrates that intelligent control can be retrofitted to existing appliances through non-invasive sensing and external actuation, rather than requiring complete device redesign. The SCT013 clamp-on current sensor and relay-based load switching enable safety monitoring without modifying the hair dryer's internal circuitry, suggesting a pathway for upgrading legacy appliances with smart capabilities.

However, the complexity of the final system—ESP32 microcontroller, external ADC, multiple sensors, Python control software, API integrations, and web dashboard—raises questions about commercial viability for low-cost consumer devices. Future work should investigate whether similar functionality can be achieved with integrated sensor modules or simplified control algorithms that reduce component count and software complexity.

Educational Value:

From a pedagogical perspective, this project successfully integrated concepts from multiple engineering domains: embedded systems programming (Arduino firmware), high-level software development (Python, FastAPI, Streamlit), electrical measurements (RMS current, temperature sensing), control theory (threshold enforcement, hysteresis), system integration (serial communication, API consumption), and user experience design (interface development, feedback mechanisms).

The iterative refinement process—from initial component testing through calibration errors to final validation—mirrors real-world product development cycles, providing valuable experience in debugging, root cause analysis, and systematic troubleshooting that extends beyond theoretical coursework.

In conclusion, this project validates the technical feasibility and practical value of intelligent, user-aware control systems for household appliances. While commercial deployment would require further engineering to reduce cost and improve robustness, the prototype successfully demonstrates measurable improvements in safety monitoring and energy efficiency awareness, advancing the vision of context-aware smart home devices that adapt to individual user needs and environmental conditions.

References

[1] Xiaomi, *Xiaomi High-speed Ionic Hair Dryer: Technical Specifications*. Xiaomi, 2023. Online product specification page.

[2] YHDC, *SCT-013 Non-Invasive AC Current Transformer*. YHDC, 2018. Datasheet.

[3] DFRobot, *Gravity: Digital Relay Module (Arduino & Raspberry Pi Compatible) (SKU: DFR0473)*. DFRobot, 2023. Online hardware specification and user documentation.

[4] Texas Instruments, *ADS1115 16-Bit Analog-to-Digital Converter with I^2C Interface*. Texas Instruments, 2015. Datasheet.

[5] DFRobot, *Gravity Expansion Shield for FireBeetle 2*. DFRobot, 2023. Online hardware specification and pinout documentation.

[6] DFRobot, *FireBeetle ESP32 IoT Microcontroller (V3.0)*. DFRobot, 2023. Online hardware specification and user documentation.

[7] National Semiconductor, *LM35 Precision Centigrade Temperature Sensors*. National Semiconductor, 2000. Datasheet.

[8] DFRobot, *Digital White LED Light Module (SKU: DFR0021)*. DFRobot, 2023. Online hardware specification and user documentation.